L3 Agentic AI Project Submission

Project Title

Enterprise Knowledge Assistant using LangGraph, RAGAS, and MCP

Problem Statement

Organizations store information across multiple systems such as knowledge repositories, policy documents, ticketing systems, CRM applications, and project management tools. Employees often spend significant time searching for accurate and up-to-date information from these disconnected sources.

Build an AI-powered Enterprise Knowledge Assistant using LangGraph that retrieves relevant information, generates an answer, evaluates the quality of that answer, and interacts with at least one external system through MCP.

The solution must demonstrate the concepts covered during the L3 Agentic AI training, including LangGraph, RAG, RAG Evaluation using RAGAS, and MCP Server integration.

Mandatory Technical Requirements

1. LangGraph Multi-Agent Design

The solution must be implemented using the LangGraph framework.

The implementation should include a minimum of three nodes acting as agents:

- Retriever Agent
- Response Agent
- Evaluator Agent

Keep the flow linear: Retrieve → Respond → Evaluate → End.

A conditional edge that re-triggers retrieval when RAGAS scores are low may be implemented for bonus credit.

2. RAG Implementation

The application must:

- Load documents from a knowledge source
- Perform chunking
- Generate embeddings
- Store embeddings in a vector database
- Retrieve relevant context
- Generate responses using retrieved context

3. RAGAS Evaluation

The application must evaluate generated responses using RAGAS.

Mandatory Metrics:

- Faithfulness
- Answer Relevancy

Bonus Metrics:

- Context Precision
- Context Recall

The application must display the evaluation scores and provide a brief interpretation of the results.

4. MCP Server Integration

The application must integrate with at least one MCP Server.

Examples include:

- Filesystem MCP Server (recommended starting point)
- SQLite MCP Server
- GitHub MCP Server
- Custom MCP Server
- Jira MCP Server

The MCP integration must be actively used by one or more LangGraph nodes during execution.

5. Observability

The application must make the graph's execution path visible end-to-end.

- Enable LangSmith tracing, or
- Log each node's inputs and outputs manually if LangSmith is not available

A single trace or log screenshot showing all nodes firing in order satisfies this requirement.

Expected Deliverables

Each associate must submit ALL of the following items.

1. Complete Source Code

Submit the complete project source code.

Before submission:

- Remove the virtual environment folder (venv)
- Remove all .env files
- Remove API keys, tokens, secrets, and credentials
- Ensure the project can be executed using the provided documentation

If source code is missing or incomplete, the submission will not be evaluated.

2. Application Execution Evidence

Submit screenshots demonstrating:

- Application startup
- Graph execution trace (node-by-node)
- RAGAS evaluation results
- Final output generated by the application

A minimum of 3–5 screenshots is required.

3. Project Documentation (README)

Provide a short README containing:

- Architecture overview (a simple or hand-drawn diagram is acceptable)
- Setup instructions — environment, dependencies, configuration
- Execution steps — how to run it, sample input, expected output
- RAG design — document source, chunking strategy, embedding model, vector database used
- LangGraph design — nodes created, node responsibilities, graph flow
- MCP integration — server used, tools exposed, use case implemented

- RAGAS evaluation — metrics collected, results, brief analysis

Evaluation Criteria

Criteria	Weightage
LangGraph Multi-Agent Design	30%
RAG Implementation	25%
RAGAS Evaluation	20%
MCP Integration	15%
Documentation	10%

Submission Guidelines

Submit the following:

1. Complete Source Code (without .env and virtual environment folders)
2. Project Documentation (README)
3. Application Screenshots

Any submission missing one or more of the above deliverables will be considered incomplete and may not be evaluated.

Submit all deliverables to:

Sharad Rajore

Email: sharad.rajore@zensar.com